

ArbiScan

Live Crypto Arbitrage Tracker

Full-Stack Implementation Plan & Resume Guide

Stack: Node.js · React · WebSockets · SQLite · Tailwind CSS

Estimated build time: 20 weeks part-time · 8 phases

1. Project overview

ArbiScan is a real-time web dashboard that monitors cryptocurrency prices across multiple exchanges simultaneously and surfaces arbitrage opportunities — moments when the same coin is priced lower on one exchange than another. The system ingests live price feeds via WebSocket connections, runs a fee-aware spread calculation engine, and pushes detected opportunities to a browser dashboard that updates instantly.

This is not a trading bot. ArbiScan is a data visualisation and monitoring tool. It does not place orders or move funds. This makes it legal to build and safe to demonstrate to employers.

Why this project stands out on a resume

Most student projects are CRUD apps (create, read, update, delete a database). ArbiScan demonstrates real-time systems engineering — WebSocket multiplexing, event-driven architecture, latency-sensitive data pipelines, and financial domain knowledge. These are skills that backend and full-stack employers actively look for.

1.1 What the system does

- Connects simultaneously to Binance, Coinbase, Kraken, and OKX via persistent WebSocket connections
- Normalises ticker data from each exchange into a common format
- Calculates spread between buy price on exchange A and sell price on exchange B, after fees
- Broadcasts detected opportunities in real time to all connected browser clients
- Displays a live-updating dashboard with filtering, sorting, and fee breakdown
- Logs historical opportunities to a SQLite database for the history tab

1.2 Architecture summary

The system has five layers that data flows through in order:

```
Exchange WebSockets → Backend aggregator → Arbitrage engine
→ WebSocket server → React dashboard
```

Each layer has a single responsibility. The aggregator only normalises data. The engine only calculates spreads. The WebSocket server only broadcasts. This separation makes the system easy to debug, test, and extend.

2. Full technology stack

Technology	Layer	Purpose in this project
<code>Node.js</code>	Backend	Runtime for the server — handles WebSocket connections to exchanges and to browsers
<code>Express.js</code>	Backend	HTTP server for the REST API (historical data, filter endpoints)
<code>ws (npm)</code>	Backend	WebSocket library — used both as client (to exchanges) and server (to browsers)
<code>ccxt</code>	Backend	Normalises exchange APIs — handles Binance/Coinbase/Kraken/OKX differences automatically
<code>better-sqlite3</code>	Backend	Logs detected opportunities to a local SQLite file for the history tab
<code>dotenv</code>	Backend	Loads API keys and config from .env file — keeps secrets out of source code
<code>React</code>	Frontend	UI framework — re-renders the dashboard when new opportunity data arrives
<code>Tailwind CSS</code>	Frontend	Utility-first CSS — dark theme, monospace numbers, responsive layout
<code>Recharts</code>	Frontend	Spread % sparkline charts in the detail drawer
<code>SQLite</code>	Database	Single-file database for opportunity history — zero configuration needed
<code>Vercel</code>	Deployment	Hosts the React frontend — free tier, one-command deploy
<code>Railway</code>	Deployment	Hosts the Node.js backend — free tier, persistent WebSocket support

3. Implementation phases

Follow these phases in order. Each phase has a concrete deliverable you can test before moving to the next. Do not skip phases — each one builds directly on the last.

Phase	Focus	Duration	Key deliverable
Phase 1	Web fundamentals	4 weeks	Static HTML page with a price table you built yourself
Phase 2	Node.js backend	3 weeks	Express server with two REST routes returning mock JSON
Phase 3	WebSockets	2 weeks	Node server pushing mock prices to a browser every second
Phase 4	Exchange APIs	2 weeks	Live BTC price from Binance logged to your terminal
Phase 5	Arbitrage engine	2 weeks	Console output showing real spread % between two exchanges
Phase 6	React frontend	4 weeks	Live-updating dashboard table connected to your backend
Phase 7	Polish & SQLite	2 weeks	History tab, filters working, fee breakdown drawer
Phase 8	Deploy	1 week	Live URL you can put on your resume

4. Phase-by-phase details

Phase 1 — Web fundamentals (4 weeks)

Goal: Understand how browsers render HTML and respond to user interaction before adding any backend complexity.

What to learn

- HTML structure: elements, attributes, semantic tags (header, main, table, section)
- CSS: box model, flexbox, grid, colors, fonts. You will use Tailwind later but learn raw CSS first
- JavaScript basics: variables, functions, arrays, objects, loops, DOM manipulation
- How the browser's developer tools work (Console, Network, Elements tabs)

Deliverable

A static HTML file with a hardcoded price table (no backend). Style it with basic CSS. Add a button that sorts the table by spread when clicked, using JavaScript. This proves you understand the fundamentals before adding complexity.

Resources

- The Odin Project (free, project-based, highly recommended)
- MDN Web Docs for any specific reference

Phase 2 — Node.js backend (3 weeks)

Goal: Run JavaScript on a server, not just in a browser. Build two REST API endpoints that return JSON.

What to learn

- How Node.js differs from browser JavaScript (no DOM, has filesystem access)
- `async/await` and Promises — critical for everything that follows
- npm: installing packages, understanding `package.json`
- Express.js: routing (`app.get`, `app.post`), middleware, request and response objects
- CORS: why it exists and how to enable it so React can call your API
- Environment variables with `dotenv`

Deliverable

An Express server with two routes: `GET /opportunities` (returns array of mock arbitrage objects) and `GET /history` (returns an empty array for now). Test both routes using a browser or the VS Code REST Client extension.

Key concept: `async/await`

```
async function getPrice(exchange, pair) {
  const response = await fetch(exchange.url + pair);
  const data = await response.json();
  return data.price;
}
```

Phase 3 — WebSockets (2 weeks)

Goal: Replace the request-response model with a persistent connection that pushes data to the browser.

What to learn

- The WebSocket lifecycle: open, message, error, close

- The ws npm library: creating a server, accepting connections, broadcasting to all clients
- The browser's native WebSocket API (no library needed on the frontend)
- JSON.parse() on incoming messages, JSON.stringify() on outgoing messages
- The readyState check before sending to avoid crashes on closed connections
- Reconnection logic: listen for close event, setTimeout to reconnect after 3 seconds

Deliverable

A Node.js WebSocket server that broadcasts a mock price object every 500ms. A plain HTML page that connects to it and updates a number on screen in real time. No React yet — just vanilla JS. This confirms the WebSocket pipeline works before adding exchange complexity.

Phase 4 — Exchange APIs (2 weeks)

Goal: Replace mock data with real prices from real exchanges.

What to learn

- How exchange WebSocket URLs are structured (each exchange has public docs)
- The ccxt library: install it, use ccxt.binance, ccxt.coinbase etc. to normalise responses
- How to connect to multiple exchanges simultaneously in one Node.js process
- Handling different message formats: Binance sends {b: price}, Coinbase sends {best_bid: price}
- Error handling per connection: one exchange going down must not crash the whole server

Deliverable

Your Node.js backend connects to Binance and Coinbase simultaneously. Both log BTC/USDT prices to your terminal in a normalised format: { exchange: 'Binance', pair: 'BTC/USDT', bid: 67418.20, ask: 67421.00, timestamp: 1234567890 }. That identical structure for both exchanges is the win.

Important note on API keys

No API keys required for price data

All four exchanges (Binance, Coinbase, Kraken, OKX) provide public WebSocket ticker feeds that require zero authentication. You only need API keys if you want to place trades, which this project never does. This means you can build and demo the entire project without creating exchange accounts.

Phase 5 — Arbitrage engine (2 weeks)

Goal: Write the core business logic that detects price gaps between exchanges.

What to learn

- How to store the latest price from each exchange in a JavaScript object (in-memory state)
- How to iterate over all exchange pairs and compare bid vs ask
- Fee structures: Binance 0.10%, Coinbase 0.10%, Kraken 0.16%, OKX 0.08%
- The spread calculation: $(\text{sell_price} - \text{buy_price} - \text{fees}) / \text{buy_price} * 100$
- How to set a minimum threshold (e.g. 0.3%) below which an opportunity is not worth flagging

The core formula

```
gross_spread = (best_bid_exchange_B - best_ask_exchange_A) /  
best_ask_exchange_A  
total_fees   = buy_fee_A + sell_fee_B  
net_spread   = gross_spread - total_fees  
is_opportunity = net_spread > MIN_THRESHOLD (e.g. 0.003 = 0.3%)
```

Deliverable

When a real price gap is detected between Binance and Coinbase, your terminal logs:
OPPORTUNITY: BTC/USDT | Buy Coinbase @ 67418.20 | Sell Binance @ 67491.00 | Net spread: 0.49% | Est. profit on \$1000: \$4.90. This is the entire value proposition of the product, working in the terminal before a single React component exists.

Phase 6 — React frontend (4 weeks)

Goal: Build the dashboard UI that connects to your WebSocket server and displays opportunities in real time.

What to learn

- React fundamentals: components, props, useState, useEffect
- How useEffect with an empty dependency array runs once on mount (where you open the WebSocket)
- Tailwind CSS: dark mode, monospace numbers with font-mono, colour utilities
- How to sort and filter an array in React state
- The flash animation pattern: add a CSS class on new data, remove it after 800ms

Component structure

```
App  
├─ Navbar           (logo, live status indicator)  
├─ StatBar          (4 metric cards)  
├─ MainContent  
│   ├─ OpportunitiesTable  
│   └─ FilterSidebar  
└─ DetailDrawer     (fee breakdown, shown on row click)
```

Deliverable

A working React dashboard that connects to your backend WebSocket and displays live opportunities in a table. New rows flash green when they appear. The Age column counts up in seconds and changes color from green to amber to red.

Phase 7 — Polish and SQLite (2 weeks)

Goal: Add the history tab, make all filters work, and log opportunities to a database.

What to add

- Install better-sqlite3: `npm install better-sqlite3`
- Create an opportunities table with columns: id, pair, buy_exchange, sell_exchange, net_spread, est_profit, detected_at
- Insert a row every time the engine detects an opportunity
- Add a GET /history endpoint in Express that queries the last 100 rows
- Add a History tab to the React frontend that fetches and displays this data
- Wire up the filter sidebar: exchange checkboxes, minimum spread slider, pair toggles

SQLite setup (20 lines of code)

```
import Database from 'better-sqlite3';
const db = new Database('arbitrage.db');

db.exec(`CREATE TABLE IF NOT EXISTS opportunities (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  pair TEXT, buy_exchange TEXT, sell_exchange TEXT,
  net_spread REAL, est_profit REAL,
  detected_at INTEGER
)`);

const insert = db.prepare(
  'INSERT INTO opportunities VALUES (?, ?, ?, ?, ?, ?, ?)'
);
```

Phase 8 — Deploy (1 week)

Goal: Get a live URL you can include on your resume and LinkedIn.

Frontend (Vercel)

- Push your React app to a GitHub repository

- Connect the repo to Vercel at vercel.com — takes about 5 minutes
- Vercel auto-deploys on every git push
- You get a free URL like arbiscan.vercel.app

Backend (Railway)

- Push your Node.js backend to a separate GitHub repository
- Connect to Railway at railway.app — free tier is sufficient
- Set your environment variables in Railway's dashboard (not in code)
- Railway gives you a persistent URL for WebSocket connections

Update your frontend

Change the WebSocket URL in your React app from `ws://localhost:8080` to your Railway URL (`wss://your-app.railway.app`). Redeploy. Your dashboard is now live.

5. Resume and interview guide

5.1 How to list it on your resume

Resume entry — copy and adapt this

ArbiScan | Live Crypto Arbitrage Tracker | github.com/yourname/arbiscan | arbiscan.vercel.app
Full-stack real-time web application that monitors cryptocurrency prices across 4 exchanges simultaneously and surfaces arbitrage opportunities after fees. Built with Node.js WebSocket server, React dashboard, and SQLite. • Engineered a real-time data pipeline ingesting 100+ price ticks/second via persistent WebSocket connections to Binance, Coinbase, Kraken, and OKX simultaneously • Implemented a fee-aware spread calculation engine with sub-100ms latency from price update to browser notification • Designed a React dashboard with live row flash animations, age-based staleness indicators, and real-time filter controls • Persisted opportunity history in SQLite with a REST API serving the history tab

5.2 What skills this demonstrates

Skill area	What ArbiScan proves you can do
Real-time systems	Handle high-frequency streaming data without blocking; broadcast to multiple concurrent clients
Event-driven architecture	Design systems around events and callbacks rather than sequential request-response
Full-stack integration	Connect a React frontend to a Node.js backend via WebSocket — not just REST

Financial domain knowledge	Understand exchange mechanics, fee structures, and spread calculations
Data pipeline design	Ingest, normalise, transform, and deliver data through a multi-stage pipeline
Database design	Schema creation, parameterised queries, pagination — even if SQLite is simple
Deployment	CI/CD concepts, environment configuration, separating frontend from backend hosting

5.3 Likely interview questions and how to answer them

"Why WebSockets instead of just polling?"

Arbitrage windows last 1-3 seconds on average. Polling every 2 seconds means you would miss most opportunities entirely. WebSockets give sub-100ms latency from price update to browser. More importantly, I am receiving 100+ price updates per second across 4 exchanges simultaneously — polling that volume would hammer the exchange APIs and get me rate-limited immediately.

"How do you handle an exchange going offline?"

Each exchange connection has its own error handler. When the close event fires, I log the disconnection and attempt reconnection after a 3-second delay with exponential backoff. The other exchange connections continue running unaffected. The frontend shows a per-exchange status indicator so the user knows if one feed has gone stale.

"Why SQLite and not PostgreSQL?"

SQLite is a single file with zero configuration, which is appropriate for a solo project with low write volume — roughly one insert per detected opportunity. The schema is simple and read patterns are straightforward. If I needed concurrent writes from multiple backend instances or complex relational queries, I would switch to PostgreSQL. I chose the simplest tool that solves the actual problem.

"How accurate are the arbitrage signals?"

The signals show real price gaps but are not guaranteed profit opportunities. They do not account for order book depth (the gap might close before your order fills), withdrawal delays between exchanges (which can take hours and expose you to price movement risk), or slippage on large orders. The tool is designed as a market observation dashboard, not a trading signal generator.

5.4 Stretch goals for extra impact

If you want to push this further, each of these adds meaningfully to the story you can tell in interviews:

- Add a second coin (ETH, SOL) and multi-pair tracking to show the system scales horizontally
- Add a profit simulation — let the user enter a trade size in USD and see estimated profit after fees on that amount
- Add exchange latency monitoring — track how long each exchange's WebSocket takes to deliver updates and surface this in the UI
- Add email or Telegram alerts when a spread exceeds a user-defined threshold
- Write unit tests for the arbitrage engine using Jest — employers are very impressed by tested financial logic
- Add TypeScript to the backend — shows maturity and makes the codebase more interview-ready

6. Done-when checklist

Use this to track your progress. Each item maps to a phase. You are ready to add this project to your resume when all items are checked.

1. Phase 1: Static HTML price table with sort-by-spread JavaScript
2. Phase 2: Express server with /opportunities and /history routes returning JSON
3. Phase 3: WebSocket server pushing mock data; browser updates in real time
4. Phase 4: Live BTC/USDT price from two exchanges logged to terminal in normalised format
5. Phase 5: Arbitrage engine logs real spread opportunities to terminal after fees
6. Phase 6: React dashboard displays live opportunities; Age column changes colour
7. Phase 7: Filters work; history tab loads from SQLite; detail drawer shows fee breakdown
8. Phase 8: Frontend live at vercel.app URL; backend live at railway.app URL
9. GitHub: both repos are public with a clear README explaining the project
10. Resume: entry added with three quantified bullet points and live URL

Final note

Build this in public. Commit your code to GitHub at the end of every session, even when it is broken. Employers look at commit history. A project with 80 commits built over 20 weeks tells a far better story than a polished project dropped in all at once. Document your learning — a short README explaining what you built and what you learned is worth more than another feature.